



**Universidad Nacional  
Autónoma de México**



**Facultad de Ingeniería**

**Laboratorio de Arquitectura y Organización  
de Computadoras**

Grupo: 07 - Semestre: 2027-1

## **Reporte Practica 4**

Construcción de Máquinas de estados Usando Memorias  
Direccionamiento Entrada - Estado

**Fecha de Entrega**

2 de Octubre del 2026

**Profesor**

Ing. Julio Cesar Cruz Estrada

**Integrantes:**

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

## 1. Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando direccionamiento de memorias con el método de direccionamiento entrada-estado.

## 2. Desarrollo

El método de direccionamiento entrada-estado se aplica a cartas ASM en las que cada estado consulta una sola variable de entrada. A diferencia del direccionamiento por trayectoria, la dirección de la ROM depende únicamente del estado presente. La palabra almacenada contiene el código de la entrada que debe probarse, dos ligas posibles y dos conjuntos de salidas. Un selector de entrada obtiene el valor de la variable indicada con dicho valor, los selectores de liga y de salida escogen la alternativa falsa o verdadera.

Para esta práctica se implementó la carta ASM propia mostrada en la Figura[1]. Las entradas se denominaron  $A$ ,  $B$  y  $C$ , y las salidas se representaron por  $S_3S_2S_1S_0$ . La máquina tiene seis estados útiles, codificados de 000 a 101. El registro mantiene el estado presente y lo usa directamente como dirección de la ROM por ello, cada estado requiere una única palabra de memoria.

### 2.1. Carta ASM y asignaciones binarias

La carta ASM se diseñó con tres condicionales: en  $S_0$  se evalúa  $A$ , en  $S_1$  se evalúa  $B$  y en  $S_2$  se evalúa  $C$ . Los estados  $S_3$ ,  $S_4$  y  $S_5$  no dependen de una entrada, por lo que su transición se fuerza al colocar las mismas ligas y salidas en las alternativas falsa y verdadera. De esta forma el valor seleccionado no modifica el resultado.

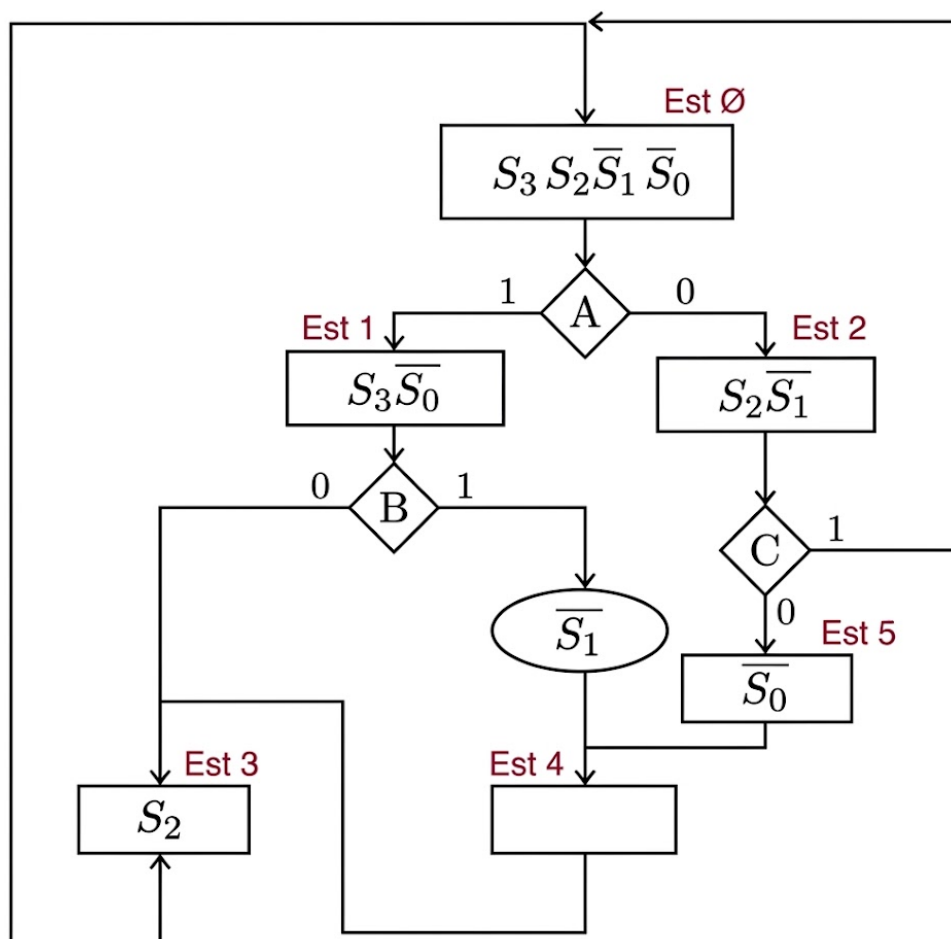


Figura 1: Carta ASM propia utilizada para el direccionamiento entrada-estado.

La asignación binaria de estados y de entradas es la siguiente:

| Estado        | $S_0$ | $S_1$ | $S_2$ | $S_3$ | $S_4$ | $S_5$ | Entrada   | $A$ | $B$ | $C$ |
|---------------|-------|-------|-------|-------|-------|-------|-----------|-----|-----|-----|
| $Q_2 Q_1 Q_0$ | 000   | 001   | 010   | 011   | 100   | 101   | $P_1 P_0$ | 00  | 01  | 10  |

El código 11 de prueba queda disponible como valor auxiliar: en `mux_prueba.vhd` se implementa como cero. En los estados sin decisión, la implementación utiliza 00, pero como sus campos falso y verdadero son idénticos, el selector es indiferente para la transición y las salidas.

## 2.2. Dimensionamiento de la memoria

Se requieren 3 bits para codificar los seis estados. Puesto que la dirección se forma sólo con el estado presente, la ROM tiene 3 bits de dirección. La palabra de memoria se divide en

cinco campos:

$$\underbrace{P_1 P_0}_{2 \text{ bits de prueba}} \quad \underbrace{LF_2 LF_1 LF_0}_{3 \text{ bits de liga falsa}} \quad \underbrace{LV_2 LV_1 LV_0}_{3 \text{ bits de liga verdadera}} \quad \underbrace{SF_3 SF_2 SF_1 SF_0}_{4 \text{ salidas falsas}} \quad \underbrace{SV_3 SV_2 SV_1 SV_0}_{4 \text{ salidas verdaderas}}.$$

Por tanto,

$$m = 2 + 3 + 3 + 4 + 4 = 16 \text{ bits}, \quad \text{ROM} = 2^n \times m = 2^3 \times 16 = 8 \times 16 \text{ bits}.$$

La capacidad total requerida es de 128 bits. Las dos direcciones no utilizadas por los seis estados de la carta, 110 y 111, se programaron como recuperación segura hacia  $S_0$ , con salidas 0011.

### 2.3. Contenido de la memoria

La Tabla[1] contiene la palabra de memoria de cada estado.  $P$  identifica la entrada que revisa `mux_prueba`,  $LF$  y  $LV$  son las ligas que se seleccionan para  $Q_{sel} = 0$  y  $Q_{sel} = 1$ , respectivamente. De la misma forma,  $SF$  y  $SV$  son las salidas para cada alternativa.

Tabla 1: Tabla de direccionamiento entrada-estado y contenido de la ROM.

| Estado       | $Q_2 Q_1 Q_0$ | Prueba $P_1 P_0$ | $LF$ | $LV$ | $SF$ | $SV$ |
|--------------|---------------|------------------|------|------|------|------|
| $S_0$        | 000           | 00               | 010  | 001  | 1100 | 1100 |
| $S_1$        | 001           | 01               | 011  | 100  | 1010 | 1000 |
| $S_2$        | 010           | 10               | 101  | 000  | 0101 | 0101 |
| $S_3$        | 011           | *                | 000  | 000  | 0111 | 0111 |
| $S_4$        | 100           | *                | 011  | 011  | 0011 | 0011 |
| $S_5$        | 101           | *                | 100  | 100  | 0010 | 0010 |
| No definidas | 110, 111      | *                | 000  | 000  | 0011 | 0011 |

Por ejemplo, cuando el registro presenta 001, la ROM entrega 01 011 100 1010 1000. El selector de prueba interpreta 01 como la entrada  $B$ . Si  $B = 0$ , la máquina avanza a 011 y presenta 1010; si  $B = 1$ , avanza a 100 y presenta 1000. Esta sola palabra reemplaza las dos localidades que habría requerido el direccionamiento por trayectoria.

### 2.4. Arquitectura e implementación en VHDL

Las Figuras[2] y [3] muestran las capturas del diagrama de bloques implementado en Quartus. El estado presente se registra en cada flanco de reloj y direcciona la ROM. El bloque `div_datos` separa los cinco campos de la palabra, `mux_prueba` evalúa la entrada seleccionada y genera `Qsel`. Finalmente, `mux_liga` y `mux_salidas` escogen, con la misma señal, la liga y las salidas que corresponden a la decisión de la carta ASM.

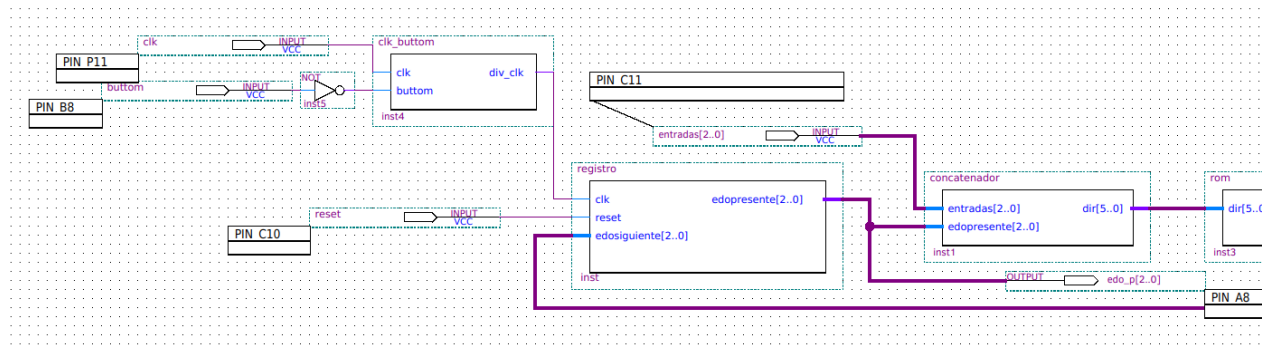


Figura 2: Primera parte del diagrama de bloques implementado en Quartus.

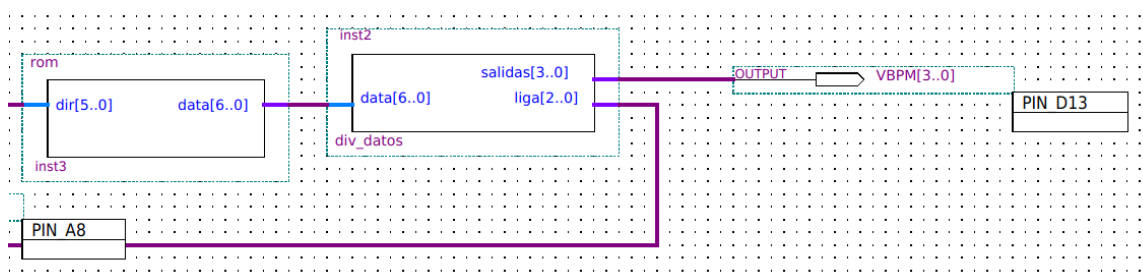


Figura 3: Segunda parte del diagrama de bloques implementado en Quartus.

## 2.5. Módulos implementados

Para los módulos implementados se reutilizaron los de la práctica anterior, el funcionamiento de cada uno de los módulos es el siguiente:

**Registro:** El módulo `registro` almacena la liga seleccionada. Su reinicio es activo en bajo, donde si `reset='0'`, fuerza el estado 000, por otro lado cuando `reset='1'`, actualiza el estado sólo en el flanco ascendente de `clk`. Esto asegura que cada transición de la máquina ocurra de manera síncrona.

Listing 1: Registro del estado presente.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity registro is
7     port(
8         clk : in std_logic;
9         reset : in std_logic;
10        edosiguiente : in std_logic_vector(2 downto 0);
11        edopresente : out std_logic_vector(2 downto 0)
12    );

```

```

13 end registro;
14
15 architecture Behavioral of registro is
16
17     signal internal_value: std_logic_vector(2 downto 0) := B"000";
18
19     begin
20         process(clk, reset, edosiguiente)
21         begin
22             if reset = '0' then
23                 internal_value <= B"000";
24             elsif rising_edge(clk) then
25                 internal_value <= edosiguiente;
26             end if;
27         end process;
28
29         process(internal_value)
30         begin
31             edopresente <= internal_value;
32         end process;
33
34     end Behavioral;

```

**ROM y división de datos:** rom.vhd declara ocho palabras de 16 bits y usa el estado presente como dirección. div\_datos.vhd no requiere reloj: separa los bits [15 : 14] como prueba, [13 : 11] como liga falsa, [10 : 8] como liga verdadera, [7 : 4] como salida falsa y [3 : 0] como salida verdadera.

Listing 2: ROM de ocho palabras para el método entrada-estado.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity rom is
6      port(
7          dir : in  std_logic_vector(2 downto 0);
8          data : out std_logic_vector(15 downto 0)
9      );
10 end rom;
11
12 architecture Behavioral of rom is
13
14     type mem is array (0 to 7) of std_logic_vector(15 downto 0);
15     signal internal_mem : mem;
16
17     begin
18
19         -----
20         -- FORMATO:
21         --
22         -- data(15 downto 14) = Prueba
23         -- data(13 downto 11) = Liga Falsa
24         -- data(10 downto 8)  = Liga Verdadera
25         -- data(7  downto 4)  = Salida Falsa
26         -- data(3  downto 0)  = Salida Verdadera
27         --
28         -- Prueba:
29         -- 00 = entrada A / I0
30         -- 01 = entrada B / I1

```

```

31  -- 10 = entrada C / I2
32  -----
33
34
35  -- Estado 0 = 000
36  -- prueba = 00
37  -- LF = 010
38  -- LV = 001
39  -- SF = 1100
40  -- SV = 1100
41  internal_mem(0) <=
42      "00" & "010" & "001" & "1100" & "1100";
43
44
45  -- Estado 1 = 001
46  -- prueba = 01
47  -- LF = 011
48  -- LV = 100
49  -- SF = 1010
50  -- SV = 1000
51  internal_mem(1) <=
52      "01" & "011" & "100" & "1010" & "1000";
53
54
55  -- Estado 2 = 010
56  -- prueba = 10
57  -- LF = 101
58  -- LV = 000
59  -- SF = 0101
60  -- SV = 0101
61  internal_mem(2) <=
62      "10" & "101" & "000" & "0101" & "0101";
63
64
65  -- Estado 3 = 011
66  -- prueba = ** (no importa)
67  -- LF = LV = 000
68  -- SF = SV = 0111
69  internal_mem(3) <=
70      "00" & "000" & "000" & "0111" & "0111";
71
72
73  -- Estado 4 = 100
74  -- prueba = ** (no importa)
75  -- LF = LV = 011
76  -- SF = SV = 0011
77  internal_mem(4) <=
78      "00" & "011" & "011" & "0011" & "0011";
79
80
81  -- Estado 5 = 101
82  -- prueba = ** (no importa)
83  -- LF = LV = 100
84  -- SF = SV = 0010
85  internal_mem(5) <=
86      "00" & "100" & "100" & "0010" & "0010";
87
88
89  -- Estado 6 = 110
90  -- No aparece especificado en la tabla.
91  -- Valor seguro: regresar al estado 000 con salida 0011.
92  internal_mem(6) <=
93      "00" & "000" & "000" & "0011" & "0011";

```

```

94
95
96  -- Estado 7 = 111
97  -- Sí aparece en la tabla
98  -- prueba = **
99  -- LF = LV = 000
100  -- SF = SV = 0011
101  internal_mem(7) <=
102    "00" & "000" & "000" & "0011" & "0011";
103
104
105  -----
106  -- Lectura de la ROM
107  -----
108  process(dir)
109  begin
110    data <= internal_mem(to_integer(unsigned(dir)));
111  end process;
112
113 end Behavioral;

```

Listing 3: Separación de los campos de la palabra de 16 bits.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity div_datos is
5    port(
6      data      : in  std_logic_vector(15 downto 0);
7
8      prueba    : out std_logic_vector(1 downto 0);
9
10     liga_f     : out std_logic_vector(2 downto 0);
11     liga_v     : out std_logic_vector(2 downto 0);
12
13     salidas_f  : out std_logic_vector(3 downto 0);
14     salidas_v  : out std_logic_vector(3 downto 0)
15   );
16 end div_datos;
17
18 architecture Behavioral of div_datos is
19 begin
20
21   prueba    <= data(15 downto 14);
22
23   liga_f    <= data(13 downto 11);
24   liga_v    <= data(10 downto 8);
25
26   salidas_f <= data(7 downto 4);
27   salidas_v <= data(3 downto 0);
28
29 end Behavioral;

```

**Selección de entrada, liga y salidas.** mux\_prueba traduce los códigos 00, 01 y 10 a A, B y C. El resultado Qsel alimenta simultáneamente a mux\_liga y mux\_salidas: con cero se seleccionan LF y SF; con uno, LV y SV. Esta selección compartida preserva la correspondencia entre las ramas de la carta ASM, la transición de estado y las salidas.

Listing 4: Selector de la entrada que debe probarse.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux_prueba is
7      port(
8          prueba: in std_logic_vector(1 downto 0);
9          I0: in std_logic;
10         I1: in std_logic;
11         I2: in std_logic;
12         Qsel: out std_logic
13     );
14 end mux_prueba;
15
16 architecture Behavioral of mux_prueba is
17 begin
18     process(prueba, I0, I1, I2)
19     begin
20         if prueba = "00" then
21             Qsel <= I0;
22         elsif prueba = "01" then
23             Qsel <= I1;
24         elsif prueba = "10" then
25             Qsel <= I2;
26         else
27             Qsel <= '0';
28         end if;
29     end process;
30 end Behavioral;

```

Listing 5: Selector de liga falsa o verdadera.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity mux_liga is
7      port(
8          Qsel: in std_logic;
9          LF: in std_logic_vector(2 downto 0);
10         LV: in std_logic_vector(2 downto 0);
11         Liga: out std_logic_vector(2 downto 0)
12     );
13 end mux_liga;
14
15 architecture Behavioral of mux_liga is
16 begin
17     process(Qsel, LF, LV)
18     begin
19         if Qsel = '0' then
20             Liga <= LF;
21         elsif Qsel = '1' then
22             Liga <= LV;
23         end if;
24     end process;
25 end Behavioral;

```

Listing 6: Selector de salidas falsas o verdaderas.

---

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity mux_salidas is
5      port (
6          Qsel : in  std_logic;
7          SF   : in  std_logic_vector(3 downto 0);
8          SV   : in  std_logic_vector(3 downto 0);
9          Salida : out std_logic_vector(3 downto 0)
10     );
11 end mux_salidas;
12
13 architecture Behavioral of mux_salidas is
14 begin
15
16     process(Qsel, SF, SV)
17     begin
18         if Qsel = '0' then
19             Salida <= SF;
20         else
21             Salida <= SV;
22         end if;
23     end process;
24
25 end Behavioral;

```

**Reloj por botón.** clk\_button cuenta pulsos del reloj de la tarjeta. Después de una pulsación, produce una señal div\_clk que genera una transición controlada del registro. Sus estados internos de espera, conteo y finalización evitan que una pulsación prolongada produzca cambios no deseados de estado.

Listing 7: Acondicionamiento del reloj mediante botón.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_button is
7      port(
8          clk : in std_logic;
9          button : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_button;
13
14 architecture Behavioral of clk_button is
15
16     CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17     CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18     CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20     CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
21         pulsos
22
23     signal state: std_logic_vector(1 downto 0) := "00";
24     signal cuenta: std_logic_vector(27 downto 0) := X"0000000";
25
26 begin
27     process(clk, button)

```

```
27  begin
28      if rising_edge(clk) then
29          case state is
30              WHEN STATE_START =>
31                  if button = '1' then
32                      state <= STATE_COUNT;
33                  end if;
34              WHEN STATE_COUNT =>
35                  if cuenta >= MAX_CLK_COUNT then
36                      state <= STATE_DONE;
37                  end if;
38              WHEN STATE_DONE =>
39                  state <= STATE_START;
40              WHEN others =>
41                  state <= STATE_START;
42          end case;
43      end if;
44  end process;
45
46  process(clk)
47  begin
48      if rising_edge(clk) then
49          if state = STATE_COUNT then
50              cuenta <= cuenta + 1;
51          else
52              cuenta <= X"0000000";
53          end if;
54      end if;
55  end process;
56
57  process(state)
58  begin
59      if state = STATE_COUNT then
60          div_clk <= '1';
61      else
62          div_clk <= '0';
63      end if;
64  end process;
65  end Behavioral;
```

### 3. Simulación

La simulación de Multisim permitió visualizar el comportamiento del sistema bajo diferentes condiciones donde en las formas de onda se observan el estado presente, el vector de salidas, la señal **reset**, las entradas *A*, *B* y *C*, y el reloj. A diferencia del direccionamiento por trayectoria, la ROM no recibe las entradas como parte de su dirección, los 3 bits de **edo\_presente** seleccionan una única palabra de 16 bits. Después, el campo P1 P0 de esa palabra indica cuál entrada debe evaluarse. El valor de la entrada seleccionada se concentra en **Qsel** y se utiliza al mismo tiempo para escoger la liga y las salidas correspondientes.

Durante la prueba, **reset** se mantiene en 1, esto debido a que el reinicio se activa en bajo. Cuando **reset** se coloca en 0, el registro fuerza inmediatamente el estado 000, correspondiente a  $S_0$ , sin depender de la entrada ni de la señal de reloj.

En la Figura[4], se aplican diferentes combinaciones a las entradas mientras se observa la

respuesta del estado y de las salidas. Al iniciar en  $S_0$  (000), la palabra de la ROM indica que debe probarse  $A$ , mediante el código  $P1P0=00$ . Si  $A = 0$ , el multiplexor de liga selecciona  $LF=010$  y el siguiente estado será  $S_2$ , si  $A = 1$ , selecciona  $LV=001$  y el siguiente estado será  $S_1$ . En ambos casos se conserva la salida 1100, tal como se muestra en la Tabla[1]. Por lo tanto, el cambio de  $A$  no altera la salida en  $S_0$ , pero sí modifica la liga que será almacenada en el siguiente flanco de reloj.

Esta misma lógica la siguen los demás estados. En  $S_1$ ,  $P1P0=01$  selecciona la entrada  $B$ : con  $B = 0$  se obtiene la liga 011 hacia  $S_3$  y la salida 1010, mientras que con  $B = 1$  se obtiene la liga 100 hacia  $S_4$  y la salida 1000. En  $S_2$ , el código 10 selecciona  $C$ , las dos alternativas conducen a  $S_5$  o  $S_0$ , manteniendo la salida 0101. Estos casos permiten comprobar y visualizar que la entrada seleccionada, y no las otras entradas del sistema, controla la decisión de cada estado.

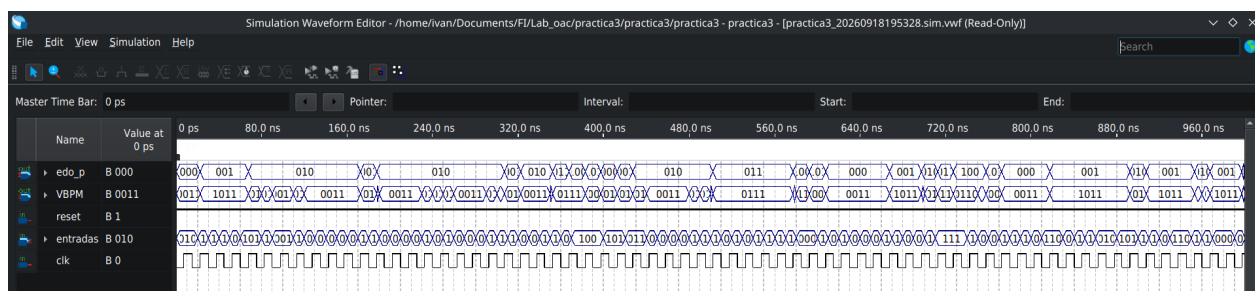


Figura 4: Primera Parte de la Simulación en Quartus.

La Figura[5] permite observar la relación temporal entre las señales. La palabra de la ROM, la entrada seleccionada y  $Qsel$  forman una ruta combinacional, de forma que cuando cambia la entrada que está siendo evaluada, pueden cambiar la liga y las salidas antes del siguiente flanco de reloj. Sin embargo, **edo\_presente** conserva su valor durante ese intervalo, ya que el registro sólo acepta la liga en el flanco ascendente.

Cuando el estado se encuentra en  $S_1$ , un cambio de  $B = 0$  a  $B = 1$  cambia la selección de la palabra de  $LF=011$ ,  $SF=1010$  a  $LV=100$ ,  $SV=1000$ . El estado mostrado continúa siendo 001 hasta el siguiente flanco posteriormente el registro presenta 100. Este comportamiento confirma que el multiplexor de liga y el multiplexor de salidas reciben la misma señal  $Qsel$ , evitando que se seleccione una transición de una rama y la salida de la otra.

En los estados  $S_3$ ,  $S_4$  y  $S_5$  no existe una decisión en la carta ASM. La ROM coloca los mismos valores en las ramas falsa y verdadera, por lo que cualquier valor de  $Qsel$  produce el mismo resultado. En consecuencia,  $S_3$  conduce a  $S_0$  con salida 0111,  $S_4$  conduce a  $S_3$  con salida 0011 y  $S_5$  conduce a  $S_4$  con salida 0010. La repetición de ambas alternativas en la palabra de memoria implementa correctamente las transiciones incondicionales.

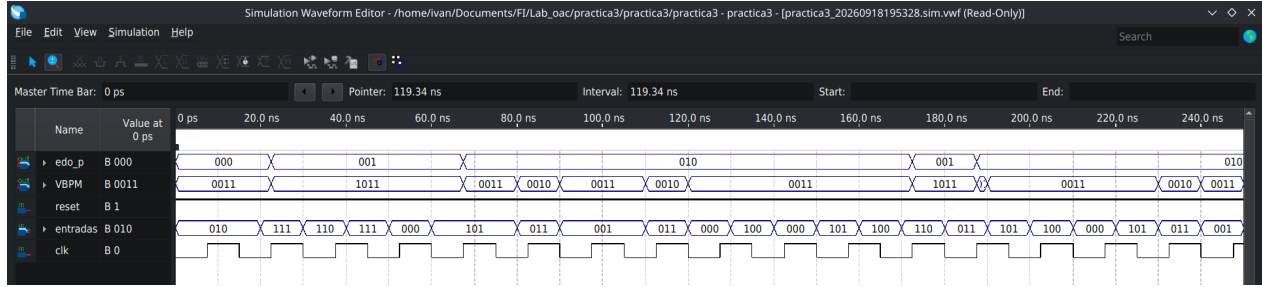


Figura 5: Detalle de las transiciones de estado y de las salidas durante la simulación.

Finalmente, en la Figura[6] se observa el tramo en el que se mantienen las entradas en valores que permiten recorrer las ramas de la carta ASM. La secuencia de estados depende de la rama seleccionada en cada estado: desde  $S_0$ ,  $A = 1$  lleva a  $S_1$ ; en  $S_1$ ,  $B = 0$  lleva a  $S_3$  o  $B = 1$  lleva a  $S_4$ ; desde  $S_2$ ,  $C = 0$  lleva a  $S_5$  y  $C = 1$  regresa a  $S_0$ . Las transiciones fijas completan los caminos restantes mediante  $S_3 \rightarrow S_0$ ,  $S_4 \rightarrow S_3$  y  $S_5 \rightarrow S_4$ .

De esta manera, la simulación permite comprobar tres aspectos del diseño. Primero, el registro se reinicia correctamente en  $S_0$ . Segundo, cada estado direcciona la palabra correcta de la ROM y prueba la entrada indicada por P1P0. Tercero, la misma decisión  $Q_{sel}$  selecciona de forma coherente la liga y las salidas, mientras que el reloj sincroniza el cambio de estado.

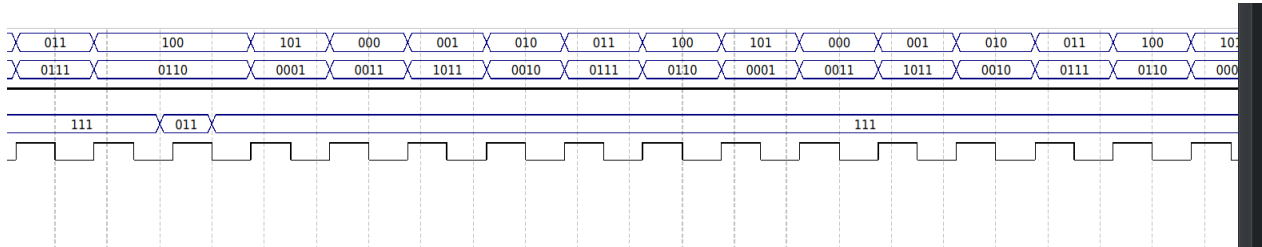


Figura 6: Parte final de la Simulación en Quartus, mostrando la secuencia de estados y salidas.

## 4. Resultados en la tarjeta FPGA

Las fotografías muestran la implementación sobre la tarjeta DE10-Lite. Los LEDs permiten observar el código de `edo_presente` y el vector `salidas`. Se documentó  $S_0$ , las dos condiciones de  $S_1$  y los estados restantes de la carta; con ello se contrastan las ramas dependientes de entrada y las transiciones incondicionales programadas en la ROM.

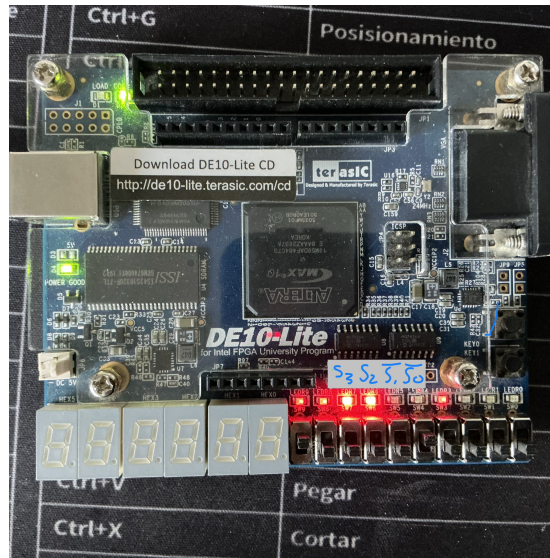
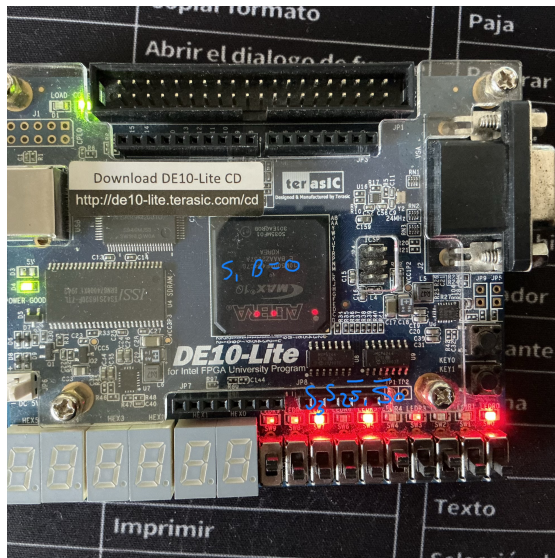
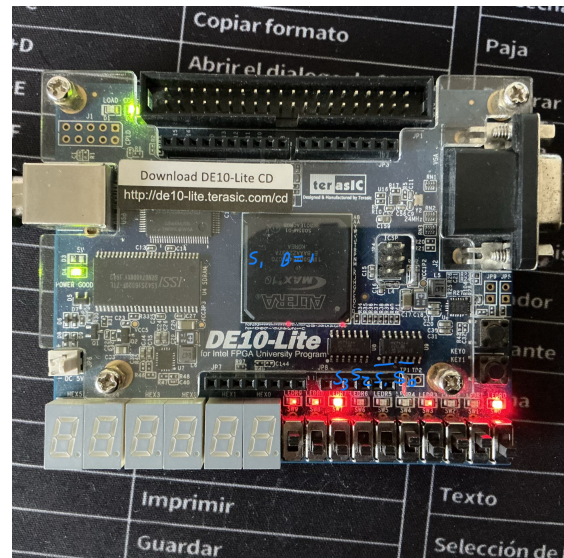


Figura 7: Evidencia en tarjeta del estado  $S_0$ , que prueba la entrada  $A$ .

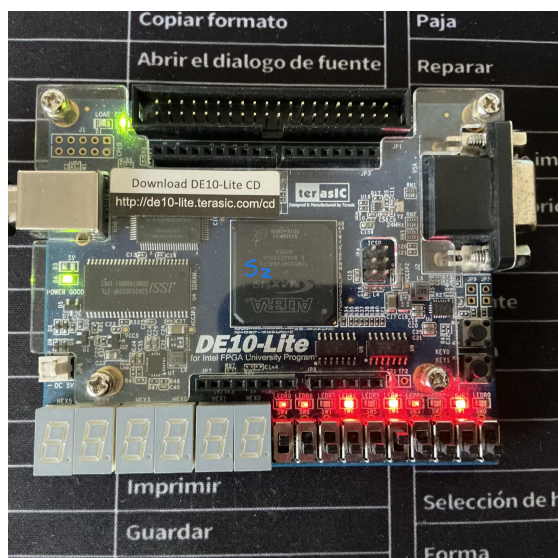


(a)  $S_1$  con  $B = 0$ : se selecciona la liga falsa hacia  $S_3$ .

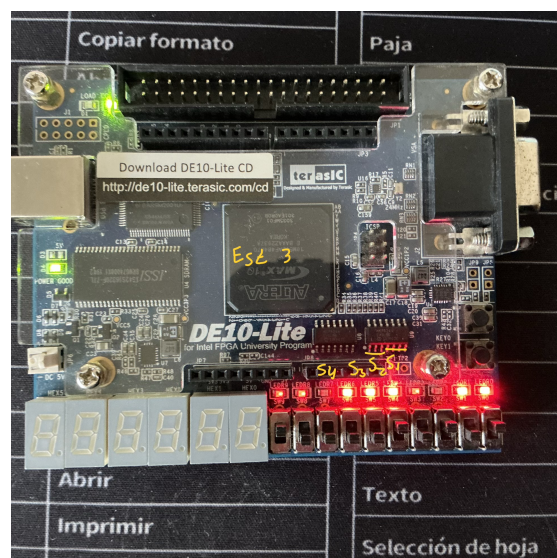


(b)  $S_1$  con  $B = 1$ : se selecciona la liga verdadera hacia  $S_4$ .

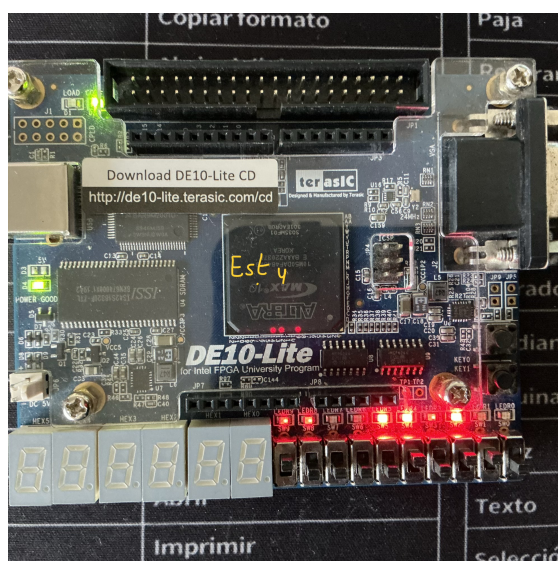
Figura 8: Verificación física de las dos ramas del estado  $S_1$ .



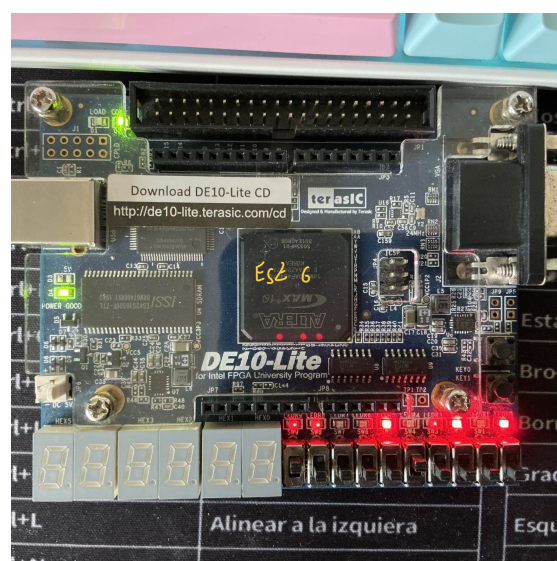
(a) Estado  $S_2$ .



(b) Estado  $S_3$ .



(c) Estado  $S_4$ .



(d) Estado  $S_5$ .

Figura 9: Evidencia en tarjeta de los estados  $S_2$ ,  $S_3$ ,  $S_4$  y  $S_5$ .

## 5. Conclusión

**Percival Ulises Espinoza Matamoros.** Gracias a la realización de esta práctica pude comprender las diferencias entre el direccionamiento por trayectoria y el direccionamiento entrada-estado, así como la implementación del direccionamiento entrada-estado en lenguaje VHDL. Al implementar este tipo de direccionamiento, se puede observar cómo una sola palabra de memoria puede contener tanto la decisión que se evalúa como las dos posibles respuestas del sistema, esto se pudo visualizar de manera más clara en la implementación física en la FPGA, así como las transiciones de estado en la simulación.

**Oscar Iván Montiel Juárez.** En este laboratorio comprendí cómo armar el direccionamiento entrada-estado para representar las decisiones y transiciones de una máquina de estados. La implementación en VHDL y las pruebas en la FPGA permitieron relacionar cada entrada con la dirección de memoria correspondiente y verificar que la salida condujera al estado esperado. Además, se puede notar que este método desperdicia menos memoria en comparación con el direccionamiento por trayectoria, ya que aprovecha mejor cada palabra almacenada.

**Amy Valentina Veraza García.** En esta práctica reforcé la relación entre el diagrama de estados, la tabla de direccionamiento y su implementación física. La simulación y la comprobación de los LEDs en la FPGA ayudaron a confirmar que las entradas  $A$ ,  $B$  y  $C$  producen las transiciones correctas y que el circuito responde de acuerdo con el diseño.